

สรุปการตั้งค่าและใช้งาน Docker Compose

คู่มือบทวนการแก้ปัญหาและการตั้งค่าสภาพแวดล้อมการพัฒนา (PERN Stack: Evaluation System)

โปรเจกต์
Evaluation System

สแต็กเทคโนโลยี
PERN (Postgres, Express, React,
Node)

เครื่องมือจัดการ DB
Adminer / pgAdmin 4

1. สรุปคำสั่ง Docker Compose ที่ใช้ในวันนี้

คำสั่งหลักๆ ที่ได้นำมาใช้แก้ไขปัญหาระบบ มีวัตถุประสงค์และการใช้งานดังนี้:

คำสั่ง (COMMAND)	วัตถุประสงค์	คำอธิบาย/บริบทที่ใช้
<code>docker compose up --build</code>	บังคับ Rebuild Container	ใช้เมื่อแก้ไขโค้ดใหม่ (เช่น เพิ่ม Route <code>/api/check-db</code>) แล้ว Docker ยังจำ Image เก่าอยู่ สั่งเพื่อสร้าง Image ใหม่จากโค้ดล่าสุด
<code>docker compose up</code>	รันระบบตามปกติ	เปิดใช้งานบริการทั้งหมดใน <code>docker-compose.yml</code> โดยใช้ Image เดิมที่เคย Build ไว้แล้ว
<code>docker compose up -d</code>	รันแบบ Background Mode	สั่งให้ Container ทำงานเบื้องหลัง ไม่ให้ยึดหน้าจอ Terminal ทำให้ใช้งาน Terminal ต่อได้
<code>docker compose watch</code>	เปิดโหมด Live Sync (Hot Reload)	ซิงค์ไฟล์ที่แก้ไขจาก VS Code เข้าไปใน Container ทันที ไม่ต้องกด rebuild หรือเปิด-ปิดใหม่
<code>docker exec -it postgres_db psql -U postgres -d evaluation</code>	เข้า CLI ฐานข้อมูล PostgreSQL	เปิดคำสั่ง <code>psql</code> เพื่อเข้าไปรันคำสั่ง SQL สร้างตาราง <code>studentTB</code> โดยตรงใน Docker

2. โครงสร้างไฟล์ `docker-compose.yml` ฉบับสมบูรณ์

ไฟล์ตั้งค่าที่รวมบริการ PostgreSQL Database, Express Backend, React Frontend และ Adminer Web UI ไว้พร้อม
ใช้งาน:

```
services:
  # 1. Database Service (PostgreSQL 16)
  postgres:
    image: postgres:16-alpine
    container_name: postgres_db
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: "1"
      POSTGRES_DB: evaluation
    ports:
      - "5432:5432"
    volumes:
      - pgdata:/var/lib/postgresql/data

  # 2. Backend Service (Express API)
  server:
    build: ./server
    container_name: express_server
```

```

    ports:
      - "5000:5000"
    environment:
      PORT: 5000
      DB_USER: postgres
      DB_PASSWORD: "1"
      DB_HOST: postgres
      DB_PORT: 5432
      DB_NAME: evaluation
    depends_on:
      - postgres
    develop:
      watch:
        - action: sync
          path: ./server
          target: /app
          ignore:
            - node_modules/
        - action: rebuild
          path: ./server/package.json

# 3. Frontend Service (React + Vite)
client:
  build: ./client
  container_name: react_client
  ports:
    - "5173:5173"
  depends_on:
    - server
  develop:
    watch:
      - action: sync
        path: ./client
        target: /app
        ignore:
          - node_modules/
      - action: rebuild
        path: ./client/package.json

# 4. Database Management UI (Adminer)
adminer:
  image: adminer:latest
  container_name: adminer_ui
  ports:
    - "8080:8080"
  depends_on:
    - postgres

volumes:
  pgdata:

```

3. สรุปวิธีใช้งานส่วนต่างๆ ที่ทำไปในวันนี้

3.1 การเชื่อมต่อ Database และจัดการตาราง studentTB

- **การสร้าง Database อัตโนมัติ:** กำหนดผ่าน `POSTGRES_DB: evaluation` ใน YAML เมื่อ Container บูตครั้งแรกจะสร้าง DB ชื่อ `evaluation` ให้ทันที
- **การสร้างตารางผ่าน Adminer (Web UI):**
 1. เปิดเบราว์เซอร์ไปที่ `http://localhost:8080`

2. เข้าสู่ระบบด้วยค่า: System= PostgreSQL , Server= postgres , Username= postgres , Password= 1 , Database= evaluation

3. ไปที่เมนู **SQL command** เพื่อวางสคริปต์ SQL สร้างตาราง หรือไปที่เมนู **Import** เพื่อนำเข้าไฟล์ .sql

• **การดูข้อมูลในตารางบน Adminer:** คลิกที่เมนู `select studentTB` ที่เมนูด้านซ้ายมือ เพื่อดูรายการข้อมูลทั้งหมดในตาราง

ข้อควรระวังในการ Import ไฟล์ SQL เข้า Adminer

- **Error "PGDMP":** เกิดจากไฟล์ที่นำเข้าเป็น Binary Dump (Custom format) แก้โดยการ Export จาก pgAdmin เป็น **Plain Text (.sql)** เท่านั้น
- **Error "syntax error at or near "":** เกิดจากคำสั่ง meta-command ของ psql เช่น `estRICT` ให้ลบบรรทัดที่ขึ้นต้นด้วย \ ออกก่อน Import
- **Error "unrecognized configuration parameter transaction_timeout":** เกิดจากไฟล์ Dump มาจาก PostgreSQL 18 แต่ Docker รุ่น Postgres 16 ให้ลบบรรทัด `SET transaction_timeout = ...` ออก หรือยกเลิกการติ๊ก **Stop on error**

3.2 การเชื่อมต่อ pgAdmin ในเครื่องเข้ากับ Docker PostgreSQL

สามารถใช้ pgAdmin 4 ในเครื่องเชื่อมต่อเข้ามาได้โดยตั้งค่า Connection ดังนี้:

- **Host name/address:** localhost
- **Port:** 5432
- **Maintenance database:** evaluation
- **Username:** postgres | **Password:** 1

3.3 การพัฒนาแบบไม่ต้องสั่ง build ใหม่บ่อยๆ (Hot Reload)

สามารถเปิดใช้ฟีเจอร์ **Compose Watch** โดยรับคำสั่ง `docker compose watch` ซึ่งระบบจะคอยตรวจจับการแก้ไขไฟล์ใน VS Code แล้วทำการซิงค์เข้าไปยัง Container และสั่งรีเฟรชหน้าเว็บ/รีสตาร์ท Express ให้แบบ Real-time ทันที